Materia: Programación III

UNIDAD 1 A 4 · DEL OBJETO A LA PERSISTENCIA

TRABAJO PRÁCTICO

Veterinaria — trabajo práctico breve con FastAPI y PostgreSQL

Objetivos

El trabajo integra, en un proyecto chico, lo desarrollado en los capítulos 1 a 4:

- los fundamentos de FastAPI (capítulo 1);
- los contratos y el CRUD (capítulo 2);
- la ejecución asincrónica sin bloqueos (capítulo 3);
- la persistencia relacional con SQLModel y PostgreSQL, con sus relaciones (capítulo 4).

Al finalizar, el proyecto entregado debe demostrar que el alumno puede:

1. Construir una API asincrónica con parámetros tipados, dependencias declaradas con `Depends()` y el motor de la base creado en el ciclo de vida de la aplicación.
2. Separar los modelos de tabla de los contratos de entrada y de salida, y responder con los códigos HTTP correctos.
3. Traducir al esquema relacional un uno a uno, una agregación, una asociación, una composición y un muchos a muchos con datos propios, justificando de qué lado va cada clave foránea.
4. Leer las relaciones con carga anticipada, sin caer en el problema N+1 ni depender de la carga perezosa.
5. Delegar en la base las garantías que la base puede dar: claves foráneas, unicidad y comprobaciones.

QUÉ QUEDA AFUERA

No se evalúan:

- autenticación ni autorización;
- repositorios ni Unit of Work;
- colas de tareas;
- migraciones con Alembic.

El esquema se crea a partir de los modelos con `SQLModel.metadata.create_all`. La lógica vive en el `service` de cada módulo, que recibe la sesión como parámetro.

Contexto

Una veterinaria de barrio necesita una API para registrar a sus clientes, sus mascotas y los turnos de atención.

- Cada dueño puede cargar una ficha con su teléfono y su dirección.
- Cada mascota figura a nombre de un dueño.
- Cada turno es de una mascota y se reserva con las prácticas que se van a hacer, por ejemplo consulta, vacuna o desparasitación.
- La veterinaria mantiene un catálogo de prácticas con su precio. Al reservar un turno, el precio de cada práctica se congela en el turno: si después cambia el precio del catálogo, los turnos ya reservados no cambian.

El dominio está elegido para que cada relación vista en el capítulo 4 aparezca una vez, con una razón concreta:

Relación	Entre	Por qué es de ese tipo
Uno a uno	Dueño y ficha	Un dueño tiene como máximo una ficha: clave foránea más unicidad (sección 4.3)
Agregación	Dueño y mascota	La mascota existe por sí misma y puede cambiar de dueño; mientras un dueño tenga mascotas a su nombre, no se puede borrar
Asociación	Turno y mascota	El turno conoce a la mascota que se atiende, pero ninguno es parte del otro
Composición	Turno y sus prácticas	Una práctica reservada no tiene sentido sin su turno: nace con él y se borra con él
Muchos a muchos con datos	Turno y práctica	Un turno lleva varias prácticas y una práctica figura en muchos turnos; el precio cobrado es de la relación, no de la práctica

Requisitos técnicos

Componente	Versión mínima	Uso
Python	3.12	Lenguaje
FastAPI	0.111	Framework web; todos los handlers con `async def`
SQLModel	0.0.24	Modelos de tabla y contratos
SQLAlchemy	2.0, asincrónico	Motor, sesión y carga anticipada
asyncpg	0.30	Controlador asincrónico de PostgreSQL
PostgreSQL	16	Base de datos
python-dotenv	1.0	Lectura del archivo `.env`
uvicorn	El que instala `fastapi[standard]`	Servidor ASGI

La dirección de la base se lee de la variable de entorno `DATABASE_URL`, definida en un archivo `.env` que no se versiona. El repositorio incluye un `env.example` con la forma del valor y sin ningún secreto.

Modelo de dominio

Los nombres de tablas y columnas son obligatorios, porque la corrección los usa. Se pueden agregar columnas, pero no quitar ni renombrar las indicadas.

La Figura 1 muestra el modelo como diagrama de clases. El rombo vacío es agregación, el rombo lleno es composición y la flecha es asociación. La tabla que sigue muestra el mismo modelo traducido al esquema.

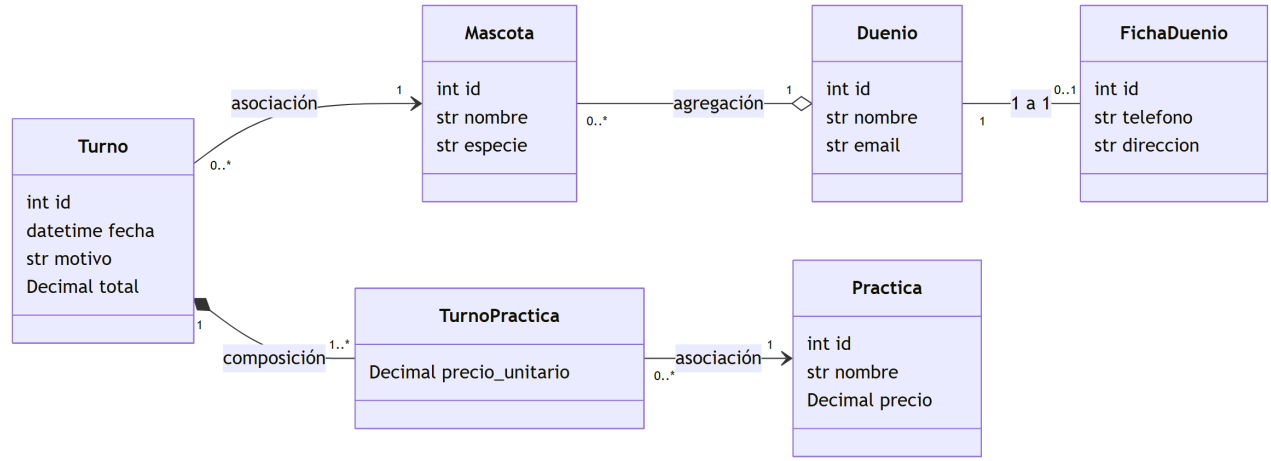


Figura 1 · Diagrama de clases. `TurnoPractica` es la clase intermedia entre turno y práctica: guarda el precio cobrado en ese turno.

Tabla	Columnas	Restricciones
`duenio`	`id`, `nombre`, `email`	`email` UNIQUE
`ficha_duenio`	`id`, `duenio_id`, `telefono`, `direccion`	`duenio_id` UNIQUE
`mascota`	`id`, `nombre`, `especie`, `duenio_id`	`duenio_id` con ON DELETE RESTRICT; índice en `duenio_id`
`practica`	`id`, `nombre`, `precio` (NUMERIC 10,2)	`nombre` UNIQUE; CHECK `precio > 0`
`turno`	`id`, `mascota_id`, `fecha` (con zona horaria), `motivo`, `total` (NUMERIC 12,2)	CHECK `total ⩾ 0`; índice en `mascota_id`
`turno_practica`	`turno_id`, `practica_id`, `precio_unitario` (NUMERIC 10,2)	Clave primaria (`turno_id`, `practica_id`); `turno_id` con ON DELETE CASCADE; CHECK `precio_unitario > 0`; índice en `practica_id`

Cada relación se declara en los dos extremos con `Relationship(back_populates="...")` (sección 4.6). `Duenio.ficha` devuelve un objeto y no una lista. El dinero se representa con `Decimal`, nunca con `float` (sección 2.7).

> **LOS ÍNDICES DE LAS CLAVES FORÁNEAS**
>
> PostgreSQL no crea solo el índice de una clave foránea (sección 4.10), por eso la tabla los pide.
>
> Hay una clave foránea que no lleva índice propio: `turno_practica.turno_id`. El `README.md` tiene que explicar en una o dos líneas por qué no hace falta. La pista está en la clave primaria de esa tabla.

Reglas de negocio

Regla	Enunciado	Garante esperado
RN-01	El email de un dueño es único.	UNIQUE en la base; 409
RN-02	El nombre de una práctica es único.	UNIQUE en la base; 409
RN-03	Un dueño tiene como máximo una ficha.	UNIQUE en `ficha_duenio.duenio_id`; 409
RN-04	El precio de una práctica es mayor que cero.	CHECK en la base y validación del DTO
RN-05	Un turno se reserva con sus prácticas: viajan dentro del cuerpo del alta del turno, y tiene al menos una.	DTO anidado; 422 si la lista está vacía
RN-06	El precio de cada práctica se copia del catálogo al reservar el turno, y el total lo calcula el servidor.	Ni el precio ni el total viajan en el DTO
RN-07	Una misma práctica no aparece dos veces en un turno.	Clave primaria compuesta de `turno_practica`; 409
RN-08	No se puede borrar un dueño que tenga mascotas a su nombre.	ON DELETE RESTRICT; 409
RN-09	Al borrar un turno se borran sus prácticas.	ON DELETE CASCADE en la base

Ninguna de estas reglas se verifica con una consulta previa del tipo "¿ya existe?". Se escribe, y si la base rechaza la escritura, el `service` atrapa el `IntegrityError`, deshace la transacción y responde 409 con un mensaje propio. La razón es la de la sección 4.10: entre la pregunta y la escritura hay una ventana en la que otra petición puede escribir lo mismo, y la única defensa real es que la base rechace la segunda.

Estructura del proyecto

La organización es por módulo funcional, con la misma estructura en todos:

Ruta	Contenido
`app/main.py`	Aplicación, lifespan y registro de routers
`app/core/config.py`	Lectura de `DATABASE_URL` desde el entorno
`app/core/database.py`	Motor, fábrica de sesiones y dependencia `get_session`
`app/modules/<módulo>/models.py`	Modelos de tabla, restricciones y relaciones
`app/modules/<módulo>/schemas.py`	DTO de entrada (`Create`, `Update`) y de salida (`Read`)

Ruta	Contenido
`app/modules/<módulo>/service.py`	Lógica, traducción de errores y lecturas con precarga
`app/modules/<módulo>/router.py`	Rutas HTTP: reciben, delegan y declaran la respuesta
`reiniciar.py`	Borra las tablas, las crea con `create_all` y carga los datos iniciales
`test/`	Archivos `.http` con los casos de prueba

Módulos mínimos: `health`, `duenios`, `mascotas`, `practicas`, `turnos` y `laboratorio`. `FichaDuenio` vive en `duenios` y `TurnoPractica` en `turnos`.

Consignas

Ejercicio 1 · Proyecto, conexión y ciclo de vida — 10 puntos

- **El motor y la sesión.** El motor asincrónico y la fábrica de sesiones se crean en el lifespan de la aplicación, se guardan en `app.state`, y el motor se cierra al apagar (sección 1.9). La sesión llega a cada handler con `Depends(get_session)`, una por petición, creada con `expire_on_commit=False` (sección 4.5).
- **La salud del servicio.** `GET /health/live` no toca la base. `GET /health/ready` ejecuta `SELECT 1` y responde 503 si la base no contesta.

Ejercicio 2 · Contratos y CRUD — 20 puntos

- **Los DTO.** Los DTO están separados de los modelos de tabla (sección 2.5), y los de alta no llevan `id`.
- **El PATCH del dueño.** Aplica sólo los campos enviados, con `exclude_unset` (sección 2.6).
- **Los códigos de respuesta** (sección 2.8):
- 201 al crear y 204 al borrar;
- 404 si el recurso no existe;
- 409 si la base rechaza la escritura;
- 422 si el cuerpo o los parámetros no cumplen el contrato.
- **El listado de mascotas.** Devuelve el envoltorio de la sección 2.9, `{ items, total, page, size, pages }`. `size` vale 20 por defecto y va de 1 a 100. Una página más allá de `pages` responde 422; la primera página siempre responde, aunque no haya resultados.

Ejercicio 3 · Modelos, restricciones y relaciones

30 puntos

- **Las tablas.** Escribir los modelos del modelo de dominio con todas sus restricciones: claves foráneas con su regla de borrado, UNIQUE, CHECK e índices.
- **La navegación.** Declararla en los dos extremos de cada clave foránea.
- **El borrado del turno.** Lo decide la base, con ON DELETE CASCADE (sección 4.3), y no el ORM. La relación `Turno.practicas` se declara de modo que el ORM no intente vaciar ni borrar esas filas por su cuenta. La pista es el parámetro `passive_deletes` de `Relationship`.
- **El `reiniciar.py`.** Deja la base en un estado conocido, con al menos:
- dos dueños, uno con ficha;
- tres mascotas;
- cuatro prácticas;
- un turno con dos prácticas.

Ejercicio 4 · Altas con relaciones: ids y datos

15 puntos

El alta de una mascota recibe el `duenio_id` de un dueño que ya existe: en la agregación viaja un id.

El alta de un turno recibe:

- el `mascota_id`: en la asociación viaja un id;
- la `fecha` y el `motivo`;
- la lista `practicas`, con el `practica_id` de cada una: en la composición viajan los datos de las partes, que nacen con el turno.

El turno se crea con sus prácticas en una sola transacción y cumple RN-05 a RN-07. Si una de las prácticas no existe, no queda guardado nada: ni el turno ni ninguna de sus prácticas.

Ejemplo del cuerpo de `POST /turnos/`. No viajan el precio de cada práctica ni el total:

```
{
  "mascota_id": 1,
  "fecha": "2026-11-12T10:30:00-03:00",
  "motivo": "Control anual",
  "practicas": [
    { "practica_id": 1 },
    { "practica_id": 3 }
  ]
}
```

Respuesta esperada, con el precio copiado del catálogo y el total calculado por el servidor:

```
{
  "id": 2,
  "fecha": "2026-11-12T13:30:00Z",
  "motivo": "Control anual",
  "total": "27000.00",
  "mascota": { "id": 1, "nombre": "Toby", "especie": "perro" },
  "practicas": [
    { "practica_id": 1, "nombre": "Consulta", "precio_unitario": "15000.00" },
    { "practica_id": 3, "nombre": "Vacuna antirrábica", "precio_unitario":
"12000.00" }
  ]
}
```

Ejercicio 5 · Lecturas sin N+1 — 15 puntos

- **La carga anticipada.** Toda respuesta que incluya datos de una relación los obtiene con carga anticipada declarada en la misma consulta: `joinedload` para relaciones «a uno» y `selectinload` para colecciones (sección 4.6).
- **Consultas fijas.** El detalle de un turno, con mascota y prácticas, se resuelve con un número fijo de consultas, que no crece con la cantidad de prácticas. Lo mismo vale para el de un dueño con su ficha.
- **Nada perezoso.** Ninguna lectura puede depender de la carga perezosa. En un contexto asincrónico, esa carga no es lenta: falla con `MissingGreenlet` (sección 3.8).

Ejercicio 6 · Asincronía sin bloqueos — 10 puntos

Todos los handlers son `async def` y ninguno ejecuta una llamada bloqueante (sección 3.6). La cátedra entrega el módulo `app/modules/laboratorio/cliente.py`. Simula la biblioteca del laboratorio que provee las vacunas, que no tiene variante asincrónica:

```python
import time


def consultar_stock(vacuna: str) -> int:
    """Biblioteca del laboratorio: sincrónica, tarda un segundo."""
    time.sleep(1)
    return 10 + len(vacuna)
```

`GET /laboratorio/stock/{vacuna}` responde `{ "vacuna": ..., "stock": ... }` llamando a esa función sin modificarla. Mientras la consulta al laboratorio está en curso, el servidor tiene que seguir atendiendo: `GET /health/live` responde de inmediato aunque haya varias consultas al laboratorio en marcha.

Endpoints de referencia

Método	Ruta	Cuerpo	Respuesta	Códigos
GET	`/health/live`	Sin cuerpo	`{"estado": "vivo"}`	200
GET	`/health/ready`	Sin cuerpo	Estado de la base	200, 503
POST	`/duenios/`	`DuenioCreate`	`DuenioRead`	201, 409, 422

Método	Ruta	Cuerpo	Respuesta	Códigos
GET	`/duenios/{id}`	Sin cuerpo	Dueño con su ficha	200, 404, 422
PATCH	`/duenios/{id}`	`DuenioUpdate`	Dueño con su ficha	200, 404, 409, 422
DELETE	`/duenios/{id}`	Sin cuerpo	Sin cuerpo	204, 404, 409
POST	`/duenios/{id}/ficha`	`FichaCreate`	`FichaRead`	201, 404, 409
GET	`/duenios/{id}/mascotas`	Sin cuerpo	Mascotas del dueño	200, 404
POST	`/mascotas/`	`MascotaCreate`	`MascotaRead`	201, 409, 422
GET	`/mascotas/`	`page` , `size`	Envoltorio con mascotas	200, 422
GET	`/mascotas/{id}`	Sin cuerpo	Mascota con su dueño	200, 404
POST	`/practicas/`	`PracticaCreate`	`PracticaRead`	201, 409, 422
GET	`/practicas/`	Sin cuerpo	Catálogo de prácticas	200
POST	`/turnos/`	`TurnoCreate` con `practicas`	Turno con mascota y prácticas	201, 409, 422
GET	`/turnos/{id}`	Sin cuerpo	Turno con mascota y prácticas	200, 404
DELETE	`/turnos/{id}`	Sin cuerpo	Sin cuerpo	204, 404
GET	`/laboratorio/stock/{vacuna}`	Sin cuerpo	Stock informado por el laboratorio	200

Casos de prueba mínimos

Los archivos `.http` de `test/` demuestran, como mínimo, los casos siguientes. Cada caso indica en un comentario la respuesta esperada, y se corre después de `python reiniciar.py`.

Caso	Qué se demuestra	Resultado esperado
C-01	Un parámetro de ruta que no es entero, por ejemplo `/duenios/abc`	422, sin ejecutar el handler
C-02	Email de dueño o nombre de práctica repetido	409 con mensaje propio
C-03	Segunda ficha para el mismo dueño	409
C-04	Mascota con un dueño inexistente, y turno con una práctica inexistente	409; del turno no queda guardado nada
C-05	Borrar un dueño con mascotas, y borrar un turno	409 en el primero; 204 en el segundo, y sus prácticas ya no están
C-06	Turno válido con dos prácticas	201, con los precios copiados y el total calculado
C-07	Detalle de un turno con una práctica y con tres	La misma cantidad de consultas en la consola (`echo=True`)
C-08	Un `UPDATE practica SET precio = 0` ejecutado directo en la base, en un archivo `.sql`	La base lo rechaza por el CHECK
C-09	Tres consultas simultáneas al laboratorio y un `GET /health/live`	El de salud responde de inmediato

Criterios de evaluación

Criterio	Peso	Qué se mira
Proyecto, conexión y ciclo de vida	10 %	Estructura, lifespan, `Depends` , `.env` y `env.example`
Contratos y CRUD	20 %	DTO separados, códigos HTTP, PATCH parcial, envoltorio de paginación
Modelos, restricciones y relaciones	30 %	Lado de cada clave foránea, uno a uno con UNIQUE, tabla intermedia con su dato, RESTRICT y CASCADE, CHECK, índices y su justificación, navegación en los dos extremos
Altas con relaciones	15 %	Ids en la agregación y la asociación; prácticas dentro del turno en la composición; todo o nada
Lecturas sin N+1	15 %	Carga anticipada adecuada a cada relación; número fijo de consultas
Asincronía sin bloqueos	10 %	Handlers asincrónicos; la biblioteca sincrónica fuera del bucle de eventos

Un proyecto que no levanta, que no crea su esquema con `python reiniciar.py` sobre una base vacía o que tiene credenciales reales en el repositorio no se corrige hasta que se subsane.

Formato de entrega

- Repositorio con el código fuente completo, `requirements.txt`, `env.example`, `reiniciar.py` y la carpeta `test/`. El archivo `.env` no se sube.
- `README.md` con los pasos para levantar el proyecto desde cero (entorno virtual, dependencias, `.env`, `python reiniciar.py` y arranque del servidor) y con la explicación del índice de `turno_practica.turno_id` pedida en el modelo de dominio.
- Evidencia del caso C-07: un fragmento de la consola con las consultas del detalle de un turno con una práctica y con tres.